

DA6401W - Assignment 5

Department of Data Science and AI, IIT Madras

April 2026

Topics covered: Batch/layer/z-score/min-max normalization, gradient clipping, hyperparameter tuning, transfer learning, convolutional neural networks, multi-channel inputs, pooling, padding, striding, ResNet, VGGNet, U-Net

Submit your solutions in one PDF file with all answers, codes, plots and outputs written/printed in that file.

1. Numerical: Normalization Dynamics and Gradient Flow in Deep CNN

Consider a convolutional neural network receiving a multi-channel input of size $64 \times 64 \times 3$. The network performs the following sequence of operations:

1. Convolution Layer 1 Filters = 16, Kernel = 7×7 , Stride = 2, Padding = 3
2. Batch Normalization
3. Max Pooling Pool size = 3×3 , Stride = 2, Padding = 1
4. Residual Block consisting of two convolution layers Kernel = 3×3 , stride = 1, padding = 1, filters = 16 Identity skip connection.
5. Global Average Pooling followed by Fully Connected layer.

Answer the following:

- (a) Compute feature map dimension after each stage.
- (b) BatchNorm statistics:

$$\mu = 12, \quad \sigma^2 = 16, \quad \gamma = 0.5, \quad \beta = -1$$

Find normalized output for activation $x = 20$.

- (c) Residual mapping produces

$$F(x) = \begin{bmatrix} 2 \\ -1 \\ 0.5 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 3 \\ -0.5 \end{bmatrix}$$

Compute block output.

- (d) Gradient clipping with threshold $c = 5$. If gradient norm is $\|g\|_2 = 25$:
- (a) clipped gradient vector expression
 - (b) new gradient norm
- (e) Min-max normalization range $[-4, 28]$. Compute normalized value for $x = 20$.

2. Programming: Transfer Learning with CNN and Effect of Normalization

In this problem you will study how normalization and transfer learning influence training stability and convergence.

1. Load the **CIFAR-10 dataset** using `torchvision.datasets.CIFAR10`.
2. Apply two different input preprocessing pipelines:
 - (i) **Min-Max normalization** to scale pixels into $[0, 1]$
 - (ii) **Z-score normalization** using dataset mean and standard deviation
3. Use a pretrained **ResNet-18** model from `torchvision.models`. Modify only the final fully connected layer to output 10 classes.
4. Freeze all convolutional layers and train only the classifier head for 5 epochs.
5. Then perform **fine-tuning** by unfreezing the last residual block and train for 10 additional epochs.
6. During training record:
 - Training loss vs epochs
 - Validation accuracy vs epochs
 - Gradient norm of the final layer
7. Repeat the experiment using a small custom CNN inspired by **VGGNet** architecture:

$$[\text{Conv}(3 \times 3) - \text{ReLU}] \times 2 \rightarrow \text{MaxPool} \rightarrow [\text{Conv}(3 \times 3) - \text{ReLU}] \times 2 \rightarrow \text{MaxPool} \rightarrow \text{FC}$$

8. Compare:
 - convergence speed with and without transfer learning
 - effect of normalization choice on gradient stability
 - final test accuracy
 9. Briefly discuss how pretrained features act as a form of **implicit regularization** and why deeper residual architectures often outperform plain CNNs.
3. Training deep neural networks often involves stabilizing both forward activations and backward gradients. Two commonly used techniques are **normalization methods** (Batch Normalization, Layer Normalization) and **gradient clipping**.

1. Batch Normalization vs Layer Normalization

Consider an input tensor $X \in \mathbb{R}^{B \times d}$, where B is the batch size and d is the feature dimension.

- (a) Write the mathematical formulation of **Batch Normalization** and clearly indicate along which dimension mean and variance are computed.
- (b) Write the formulation of **Layer Normalization** and specify the normalization dimensions.
- (c) Explain why Batch Normalization performance degrades when the batch size is very small, whereas Layer Normalization does not suffer from this issue.
- (d) In the context of sequence models (e.g., Transformers), explain why Layer Normalization is preferred over Batch Normalization.

2. Effect on Optimization Dynamics

- (a) Explain how normalization methods affect the conditioning of the optimization problem.
- (b) Discuss the role of normalization in mitigating *internal covariate shift*. Is this explanation sufficient to justify their effectiveness?

3. Gradient Clipping

During training, suppose the gradient vector $g \in \mathbb{R}^n$ has a very large norm.

- (a) Define **gradient clipping by norm** and write its mathematical formulation.
- (b) Show that after clipping with threshold c , the resulting gradient has norm at most c .
- (c) Explain how gradient clipping helps in training recurrent or very deep networks.

4. U-Net is a widely used architecture for image segmentation that combines an encoder-decoder structure with skip connections.

In this problem, you will implement and analyze a U-Net model using a real-world segmentation dataset.

1. Dataset

You are given the following dataset class based on the Oxford-IIIT Pet dataset:

```
class OxfordPetsSegmentation(Dataset):
    def __init__(self, root='.', split='train', image_size=128):
        self.dataset = OxfordIIITPet(root=root, download=True,
                                     target_types='segmentation')
        self.images = self.dataset._images
        self.masks = self.dataset._segs
        total = len(self.images)
        if split == 'train':
            self.images = self.images[:int(0.8 * total)]
            self.masks = self.masks[:int(0.8 * total)]
```

```

else:
    self.images = self.images[int(0.8 * total):]
    self.masks = self.masks[int(0.8 * total):]

self.img_transform = torchvision.transforms.Compose([
    torchvision.transforms.Resize((image_size, image_size)),
    torchvision.transforms.ToTensor()
])

self.mask_transform = torchvision.transforms.Compose([
    torchvision.transforms.Resize((image_size, image_size),
                                   interpolation=Image.NEAREST),
    torchvision.transforms.PILToTensor()
])

def __len__(self):
    return len(self.images)

def __getitem__(self, idx):
    img = Image.open(self.images[idx]).convert("RGB")
    mask = Image.open(self.masks[idx])
    img = self.img_transform(img)
    mask = self.mask_transform(mask).squeeze(0) - 1
    return img, mask.long()

```

The dataset provides:

$$X \in \mathbb{R}^{3 \times 128 \times 128}, \quad Y \in \{0, 1, 2\}^{128 \times 128}$$

corresponding to background, pet, and outline classes.

2. U-Net Architecture

Implement a U-Net with the following structure:

- Encoder:

$$3 \rightarrow 32 \rightarrow 64 \rightarrow 128$$

Each block: Conv 3×3 + ReLU + Conv 3×3 + ReLU + MaxPool

- Bottleneck:

$$128 \rightarrow 256$$

- Decoder:

$$256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 3$$

Each block: Transposed Conv + concatenation (skip connection) + Conv layers

3. Skip Connections

Implement skip connections by concatenating encoder feature maps with corresponding decoder feature maps.

4. Training

- Use Cross-Entropy loss
- Train on the training split and evaluate on the validation split
- Epochs: 25 - 30

5. Ablation Study

Train a second model where skip connections are removed (i.e., decoder receives no encoder features).

6. Evaluation

- Plot training and validation loss curves
- Report pixel accuracy or IoU
- Visualize predicted segmentation maps

Sample Predicted Segmentation Map:

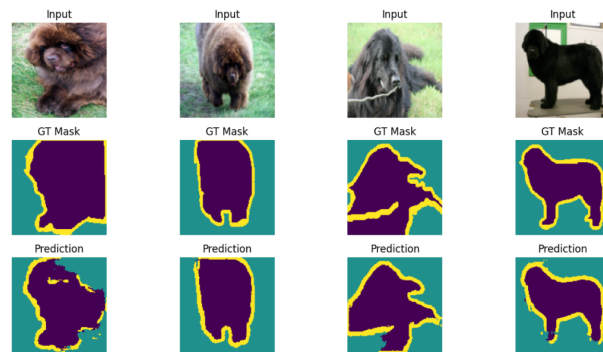


Figure 1: Image Segmentation

7. Analysis

Answer the following:

- Why do skip connections improve segmentation performance in U-Net?
- What types of errors increase when skip connections are removed?
- How does this differ from residual connections used in ResNets?

5. Numerical: Effect of Stride and Padding in Convolution

Consider an input image of size

$$64 \times 64 \times 3$$

A convolution layer is applied with the following parameters:

- Number of filters = 32
- Kernel size = 5×5

- Stride = 2
 - Padding = 2
1. Compute the spatial dimensions of the output feature map.
 2. Compute the total number of learnable parameters in the convolution layer (including bias).

6. Conceptual: Residual Connections and Optimization

Consider two neural networks with identical depth:

- Network A: Plain CNN
- Network B: Residual CNN with skip connections

A residual block computes

$$y = F(x) + x$$

Answer the following:

1. Explain why residual connections improve gradient flow during backpropagation.
2. Suppose $F(x) = 0$. What is the output of the residual block? What does this imply about training very deep networks?

7. **Programming: preprocessing techniques** In this question, you will explore how different data scaling techniques affect the convergence speed, optimization stability, and final predictive performance of a neural network. You will implement a standard regression pipeline using PyTorch/Tensorflow, apply different preprocessing techniques to the input features, and analyze the resulting learning curves.

Dataset Details

- **Dataset:** California Housing Dataset (Regression).
- **Source:** `sklearn.datasets.fetch_california_housing`
- **Data Splitting:** Perform a standard 80/20 train-test split using a random state of 42 to ensure reproducibility.

Model Architecture You must construct a Multi-Layer Perceptron (MLP) with exactly 4 hidden layers. The architecture specifications are as follows:

- **Input Layer:** Dynamically sized to match the dataset's feature count.
- **Hidden Layer 1:** 64 neurons, ReLU activation.
- **Hidden Layer 2:** 64 neurons, ReLU activation.
- **Hidden Layer 3:** 32 neurons, ReLU activation.

- **Hidden Layer 4:** 16 neurons, ReLU activation.
- **Output Layer:** 1 neuron, Linear activation (standard for regression).

Optimization & Training Routine

- **Optimizer:** Adam
- **Learning Rate:** 0.001
- **Loss Function:** Mean Squared Error (MSE)
- **Epochs:** 100

You must run the training loop three separate times, applying a different preprocessing technique to the input features each time.

- **Pipeline A (Min-Max Normalization):** Scale the input features to a range of $[0, 1]$.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Pipeline B (Z-Score Standardization):** Scale the input features to have a mean of 0 and a standard deviation of 1.

$$X_{std} = \frac{X - \mu}{\sigma}$$

- **Pipeline C (Raw Data):** Pass the raw, unnormalized input features directly into the network.

Note: Be sure to fit your scalers ONLY on the training data, and then use them to transform both the training and validation data to prevent data leakage.

Plot the validation loss curves for all three pipelines on the same graph and write your observation about which preprocessing is performing better and its justification.

8. **Programming: Hyperparameter tuning (Grid vs. Random Search)** Automate the search for optimal hyperparameters using two distinct strategies. By constraining your computational budget to exactly 9 runs per strategy, you will evaluate which method searches the hyperparameter space more effectively.

Setup & Data

- Dataset: CIFAR-100 (pixels normalized to 0-1).
- Splits: Train (40,000), Validation (10,000), Test (10,000).
- Fixed Training Settings: Adam optimizer, Categorical Crossentropy loss, 15 epochs, Batch Size 128.

Base Model Architecture

- Conv Block 1: 2x Conv2D (32 filters, 3x3, ReLU) → MaxPooling2D.

- Conv Block 2: 2x Conv2D (64 filters, 3x3, ReLU) → MaxPooling2D.
- Conv Block 3: 2x Conv2D (128 filters, 3x3, ReLU) → MaxPooling2D.
- Classification Head: Flatten → Dense (512 units, ReLU) → Dropout (Variable) → Dense (100 units, Softmax).

The Search Challenge You will tune Learning Rate (LR) and Dropout Rate using two methods. Do exactly 9 runs for each:

Strategy A (Grid Search): Test every combination of:

- LR: 0.0005, 0.001, 0.005
- Dropout: 0.2, 0.4, 0.6

Strategy B (Random Search): Sample 9 random combinations from:

- LR: Log-uniform distribution between 0.0001 and 0.01.
- Dropout: Uniform distribution between 0.1 and 0.6.

Provide a table showing the Top 3 configurations (based on Validation Accuracy) for both Strategy A and Strategy B. Include the LR, Dropout, and Loss.

Final Evaluation: Take the single best configuration from all 18 runs, train it, and report the final accuracy on the unseen Test Set.

9. Numerical: Large Kernel vs Small Kernels with Pooling

Consider an input feature map:

$$X \in \mathbb{R}^{64 \times 64 \times 3}$$

Two different designs are applied:

Design A (Single Large Convolution):

- Kernel size = 7×7
- Number of filters = 64
- Stride = 2, Padding = 3

Design B (Smaller Convolutions + Pooling):

- First layer: 3×3 , 32 filters, stride = 1, padding = 1
- Max Pooling: 2×2 , stride = 2
- Second layer: 3×3 , 64 filters, stride = 1, padding = 1

Tasks:

- (a) Compute the output feature map dimensions for:
 - Design A
 - Design B (after all layers)
- (b) Compute the total number of learnable parameters (including bias) for Design A.
- (c) Compute the total number of learnable parameters (including bias) for Design B.
- (d) Compute the ratio:

$$\frac{\text{Parameters in Design B}}{\text{Parameters in Design A}}$$

10. Programming: Visualizing Feature Maps in CNN (CIFAR-10)

In this problem, you will train a Convolutional Neural Network (CNN) on the CIFAR-10 dataset and visualize feature maps at different layers to understand how representations evolve.

Dataset:

- CIFAR-10 dataset using `torchvision.datasets.CIFAR10`
- Apply normalization to scale pixel values to $[0, 1]$

Model Architecture (Predefined CNN):

Input: (3, 32, 32)

Conv1:

Convolution: 16 filters of size (3 × 3), stride = 1, padding = 1

Activation: ReLU

Max Pooling: (2 × 2), stride = 2

Conv2:

Convolution: 32 filters of size (3 × 3), stride = 1, padding = 1

Activation: ReLU

Max Pooling: (2 × 2), stride = 2

Conv3:

Convolution: 64 filters of size (3 × 3), stride = 1, padding = 1

Activation: ReLU

Flatten

Fully Connected Layer:

Linear: $(64 \times 8 \times 8) \rightarrow 128$
Activation: ReLU

Output Layer:

Linear: $128 \rightarrow 10$ (logits)

Note: Softmax is applied implicitly inside the Cross-Entropy loss during training.

Training Setup:

- Optimizer: Adam
- Learning Rate: 0.001
- Loss Function: Cross-Entropy
- Epochs: 50

Tasks:

- (a) Train the CNN on CIFAR-10 for 50 epochs and report:
 - Training loss vs epochs
 - Test accuracy
- (b) For a sample input image, extract and visualize feature maps after:
 - Conv1
 - Conv2
 - Conv3Display at least 6 channels (feature maps) from each layer.
- (c) Report observations from feature maps:
 - What type of patterns are learned in early layers?
 - How do feature maps change in deeper layers?
- (d) Compare activations before and after pooling:
 - What changes in spatial resolution?
 - What happens to important features?
- (e) Based on your training results, comment on:
 - Does the model overfit or generalize well?
 - At which epoch does accuracy stabilize?

11. (Theoretical Question — Weight Initialization in CNNs with ReLU)

Consider the following CNN architecture trained for image classification with inputs shape $(\text{batch_size}, a, b, 3)$:

Fan-in and Fan-out Definitions:

- **Convolutional layers:** both fan_in and fan_out use the full receptive field area $k \times k$, multiplied by the respective channel count:

$$\text{fan_in} = C_{\text{in}} \times k \times k, \quad \text{fan_out} = C_{\text{out}} \times k \times k$$

- **Fully connected layers:**

$$\text{fan_in} = \text{number of input neurons}, \quad \text{fan_out} = \text{number of output neurons}$$

Layer	Type	Details	Activation
1	Conv2D	kernel 3×3 , in_ch = 3, out_ch = 64	ReLU
2	BatchNorm + MaxPool	2×2 pool	—
3	Conv2D	kernel 3×3 , in_ch = 64, out_ch = 128	ReLU
4	BatchNorm + MaxPool	2×2 pool	—
5	Conv2D	kernel 3×3 , in_ch = 128, out_ch = 256	ReLU
6	Flatten	$4 \times 4 \times 256 = 4096$	—
7	FC (Linear)	$4096 \rightarrow 1024$	ReLU
8	FC (Linear)	$1024 \rightarrow 10$	Softmax

- State the **Kaiming He Normal** initialization formula. Define every term clearly, including how fan_in is computed for a convolutional layer.
- State the **Kaiming He Uniform** initialization formula and explain how it relates to the normal variant.
- For each **convolutional layer** in the network above, compute fan_in , fan_out , the standard deviation σ (normal), and the bound a (uniform).
- For each **fully connected layer** in the network above, compute fan_in , fan_out , σ , and a .

12. (Hyperparameter Optimization Strategies)

- How does Bayesian Optimization differ from grid search and random search in its approach to hyperparameter tuning? What key advantage does it offer?
 - Why cannot Bayesian Optimization replace gradient descent for training neural networks?
 - Why cannot gradient descent be used directly for hyperparameter tuning? What property of the hyperparameter optimization problem makes it fundamentally different from model training?
13. In an image segmentation task, the goal is to separate pets from the background. The ground truth labels are provided as a tensor:

$$Y \in \mathbb{R}^{H \times W \times 1}$$

where each pixel takes a value in $\{0, 1\}$, with 1 indicating the presence of a pet and 0 indicating background.

Now consider extending this task to identify additional elements in the image such as trees, roads, and other objects.

1. Label Representation

- (a) Do you think the same labeling strategy (single-channel binary mask) would be sufficient for this extended task? Justify your answer.
- (b) If not, propose a suitable labeling strategy. Clearly describe:
 - how each pixel should be represented,
 - the shape of the label tensor.
- (c) What should be the shape of the model output for this task?

2. Loss Function

- (a) What loss function would you use for this problem? Write its mathematical form.
- (b) In practice, datasets may have imbalance between different regions (e.g., large background vs small objects). Suggest an additional loss (or modification) to address this and briefly explain why.

14. **Programming:** In this problem, you will study the effect of **gradient clipping** during **transfer learning** on the **CIFAR-100** dataset. Use a pretrained **ResNet-18** model and modify it for CIFAR-100 classification. Freeze the early layers and **unfreeze the deeper layer(s)** so that only the later part of the network is fine-tuned.

Let the gradient vector of all trainable parameters at a training step be denoted by

$$g \in \mathbb{R}^d,$$

with gradient norm

$$\|g\|_2.$$

When gradient clipping is applied with threshold c , the clipped gradient is

$$\tilde{g} = g \cdot \min \left(1, \frac{c}{\|g\|_2} \right).$$

You will run experiments with clipping thresholds

$$c \in \{5.0, 1.0, 0.5, 0.1\}.$$

1. Model Setup and Training Protocol

- (a) Describe how you would adapt pretrained ResNet-18 for CIFAR-100 classification. Clearly specify:
 - which layers are frozen,
 - which deeper layer(s) are unfrozen,

- how the final classification head is modified.
- (b) Explain how gradient clipping is applied during backpropagation. At what point in the training loop should clipping be performed?

2. Gradient Norm Measurement

- (a) Give the mathematical expression for the total gradient norm before clipping.
- (b) For a clipping threshold c , what will be the gradient norm after clipping when:
- $\|g\|_2 \leq c$,
 - $\|g\|_2 > c$?
- (c) Define a quantitative measure for the amount of gradient signal retained after clipping. One possible choice is

$$r = \frac{\|\tilde{g}\|_2}{\|g\|_2}.$$

Explain what values of r indicate mild clipping versus severe clipping.

3. Experimental Study

- (a) Train the model separately for clipping norms 5.0, 1.0, 0.5, and 0.1.
- (b) For each setting, plot:
- gradient norm before clipping vs. training step (or epoch),
 - gradient norm after clipping vs. training step (or epoch),
 - validation accuracy vs. epoch.
- (c) Compare the three runs and comment on:
- how often clipping is activated,
 - how strongly gradients are suppressed,
 - whether optimization appears stable or overly constrained.

4. Analysis of Gradient Dynamics and Performance

- (a) Based on your plots, which clipping threshold preserves the largest fraction of the gradient signal? Which one suppresses it the most?
- (b) How does the retained gradient signal relate to training and validation performance? Discuss whether aggressive clipping helps or hurts transfer learning in this setting.
- (c) Suppose the model with clipping norm 0.1 shows smooth gradient curves but worse accuracy than the model with clipping norm 1.0. Give a plausible explanation in terms of optimization dynamics.

15. Programming: Multi-Channel Convolution and Effect of Stride/Padding

In this problem, you will implement a convolution operation from scratch using NumPy to understand how multi-channel inputs interact with filters, stride, and padding.

1. Implement a function:

```
conv2d_multi(X, W, stride, padding)
```

where:

- $X \in \mathbb{R}^{C \times H \times W}$ is the input
 - $W \in \mathbb{R}^{F \times C \times K \times K}$ is the filter bank
 - stride and padding are integers
2. Apply zero-padding manually before convolution.
 3. Perform convolution operation across all channels and filters.
 4. For a fixed input:

$$X \in \mathbb{R}^{3 \times 32 \times 32}, \quad W \in \mathbb{R}^{8 \times 3 \times 3 \times 3}$$

Evaluate output feature maps under:

- Case 1: stride = 1, padding = 1
 - Case 2: stride = 2, padding = 1
 - Case 3: stride = 1, padding = 0
5. Visualize how feature map size changes across these settings.
 6. Compare:
 - computational cost (number of operations)
 - spatial information retention